

A DQN-Based Hybrid Decision Framework for Dynamic Memory Allocation

Sicheng Yang
School of Computer Science and
Engineering
Sun Yat-sen University
Guangzhou, China
yangsch@mail2.sysu.edu.cn

Jing Ma*
School of Automation and
Electrical Engineering
University of Jinan
Jinan, China
*cse_maj@ujn.edu.cn

Abstract— Dynamic memory allocation is a fundamental and critical operation in computer systems, whose efficiency directly affects program performance and memory utilization. Traditional allocation algorithms rely on fixed heuristic rules that cannot adaptively adjust decision strategies in response to workload changes, exhibiting inherent limitations in fragmentation control and long-term memory health maintenance. This paper formulates the dynamic memory allocation problem as a Markov Decision Process and proposes a DQN (Deep Q-Network)-based hybrid decision framework. Rather than directly outputting allocation addresses, the framework selects from a carefully constructed candidate block set, compressing the action space to a tractable scale while preserving decision flexibility. A reward function based on the Herfindahl-Hirschman Index (HHI) has been designed to guide the agent in learning to maintain the concentration of free memory space. Experimental results in a simulated environment demonstrate that the proposed approach significantly outperforms Worst-Fit and First-Fit in memory occupancy and service duration, achieves performance comparable to Best-Fit, and exhibits unique advantages in the memory concentration metric. This work preliminarily validates the feasibility of reinforcement learning in the memory allocation domain, providing a new perspective for building adaptive, learnable system resource management strategies.

Keywords— *Dynamic memory allocation; Deep reinforcement learning; DQN; Memory fragmentation; Resource management*

I. INTRODUCTION

Dynamic memory allocation is one of the core components of modern computer systems. From operating system kernels to user-space applications, programs dynamically request and release memory through malloc and free (or equivalent mechanisms), enabling programs to flexibly manage limited memory resources at runtime. However, this flexibility also introduces the classic problem of external fragmentation: as allocation and deallocation operations alternate, originally contiguous free memory space is progressively divided into scattered small blocks, which may eventually lead to situations where, despite sufficient total free memory, the system cannot satisfy a large contiguous allocation request, causing premature program failures or performance degradation [1]. In practical systems, dynamic memory allocation is typically handled by allocators. Widely adopted allocation strategies in both industry and academia include First-Fit, Best-Fit, and

Worst-Fit [1]. These algorithms are all based on fixed heuristic rules: First-Fit selects the first sufficiently large free block; Best-Fit selects the free block that produces the smallest remaining fragment; Worst-Fit selects the largest free block. Their respective advantages and disadvantages have been thoroughly studied: Best-Fit generally performs best in controlling fragmentation but incurs significant search overhead in highly fragmented scenarios; First-Fit is simple to implement with fast allocation speed but tends to accumulate fragments at low memory addresses; Worst-Fit preserves larger free blocks but is prone to prematurely exhausting available large contiguous spaces.

The common limitation of these traditional methods is that they lack the ability to adapt to changes in workload characteristics. Regardless of how the allocation/deallocation patterns of programs change, the algorithms always maintain the same decision logic. Once workload characteristics or optimization objectives change, the heuristic rules need to be redesigned or manually tuned—a time-consuming process that relies on expert experience.

Deep Reinforcement Learning (Deep RL) has achieved breakthrough progress in complex sequential decision-making problems [2, 3], bringing new paradigms to the field of system resource management. We argue that the following characteristics of RL make it equally applicable to the dynamic memory allocation problem. (1) Online decision-making capability: Memory allocation is essentially an online sequential decision-making problem, where the agent must make immediate choices upon each allocation request—a pattern that naturally aligns with the RL interaction paradigm. (2) Lookahead capability: Through the recursive structure of the Bellman equation, an RL agent can evaluate the long-term impact of current decisions on future states, rather than focusing solely on immediate rewards. This "foresight" is crucial for memory allocation—an allocation choice that appears optimal in the short term (e.g., minimizing immediate waste) may lead to severe memory fragmentation in the long run. (3) Adaptive capability: RL agents can adapt to the characteristics of specific workloads (such as allocation size distributions and deallocation frequencies) through continuous learning, without requiring manual algorithm redesign for each scenario.

This paper explores the application of deep reinforcement learning to the dynamic memory allocation problem. Since memory allocation is extremely sensitive to decision latency, it is infeasible to have the RL model search the entire memory address space directly. Therefore, we propose a candidate block selection hybrid scheme: traditional algorithms are responsible for filtering out a small candidate block set, while the DQN agent selects the block with the highest long-term return from this set for allocation. In essence, the agent in this paper is learning a "smarter non-greedy" allocation strategy. The main contributions of this paper are as follows:

Formalizing the dynamic memory allocation problem into a complete reinforcement learning framework, including the design of state space, action space, and reward function.

Proposing an HHI-based reward function that evaluates the "health" of free memory space from a global perspective, with desirable properties including smoothness, normalization, and concentration encouragement.

Testing the generalization ability of the RL policy under multiple request distributions, validating its robustness advantages over fixed heuristic algorithms.

II. BACKGROUND

A. Dynamic Memory Allocation Problem

Consider a memory space of size M . At discrete time steps $t = 1, 2, \dots$, the system sequentially processes two types of requests: allocation requests (for contiguous space of size s_t) and deallocation requests (freeing a previously allocated block). The allocator needs to select a sufficiently large free block for each arriving allocation request at every step, marking exactly s_t space within it as allocated—if the selected block is larger than s_t , the remainder is returned to the free list as a new, smaller free block; if no sufficiently large free block exists, the allocation fails.

After T operations, the free space may be divided into multiple non-contiguous small blocks. Let the free block set be $F = \{(a_i, l_i)\}$, where a_i is the start address and l_i is the block size. The total free space is $S_{total} = \sum l_i$. We focus on the following core metrics: Occupancy: $1 - S_{total}/M$, measuring the final utilization level of memory.

Duration: The total number of allocation requests successfully completed before the first allocation failure, measuring the allocator's persistence.

Fragmentation: $1 - \max(l_i)/S_{total}$, where lower values indicate less fragmentation [1].

Herfindahl-Hirschman Index (HHI): $\sum (l_i/S_{total})^2$, measuring the concentration of free space, where higher values indicate more concentrated free space.

B. Reinforcement Learning Fundamentals

Reinforcement learning considers the general framework of an agent interacting with an environment [4]. Q-Learning and DQN. Q-Learning [5] is a model-free RL algorithm whose core is learning the action-value function $Q(s, a)$, representing the expected cumulative reward obtainable by following the optimal policy after taking action a in state s . The optimal Q function satisfies the Bellman optimality equation. When the state space

is large, a function approximator is needed. DQN [2] employs a deep neural network as the approximator for the Q function, while introducing two key techniques to ensure training stability: Experience Replay and Target Network. Experience replay breaks the temporal correlation of time-series data through random sampling from a buffer; the target network uses a slowly-updated copy of the network to compute target Q -values, avoiding the "chasing a moving target" problem that causes training divergence.

III. SYSTEM DESIGN

A. Model Overview

The memory allocation system was modeled as an RL environment. The system maintains a memory space of size M , with a free block linked list and an allocated block dictionary as core data structures. At each step, the environment provides the Agent with a fixed-dimension state vector, receives the action selected by the Agent (candidate block index), executes the allocation operation, and returns a reward signal. The overall process is illustrated in Fig. 1.

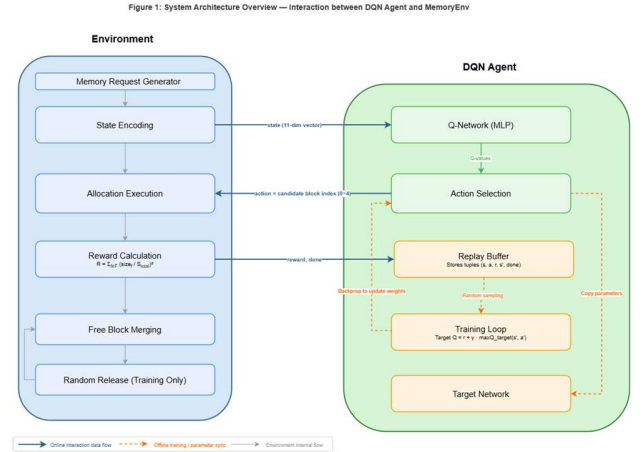


Fig. 1. System architecture overview—Interaction between DQN Agent and MemoryEnv

B. Reinforcement Learning Formulation

1) State Space. The design of the state vector faces a critical trade-off: an overly compact state may lose important information, while an overly complex state increases network size and inference latency. Considering the latency sensitivity of memory allocation, we choose a compact state encoding scheme. Let k be the number of candidate free blocks (we set $k = 5$ in this paper). For each candidate block, we record its normalized size l_i/M and normalized start address a_i/M . Additionally, the normalized current request size s_t/M is appended to the end of the state vector. The base state dimension is therefore $2k + 1 = 11$, as show in Fig. 2.

The candidate block selection strategy is: sort by start address in ascending order and take the first k free blocks whose sizes satisfy the current request. This essentially constructs the candidate set in a First-Fit manner but delegates the final decision to the RL agent. All values are normalized by dividing by M , decoupling the model from specific memory capacities.

Figure 2: State Vector Composition

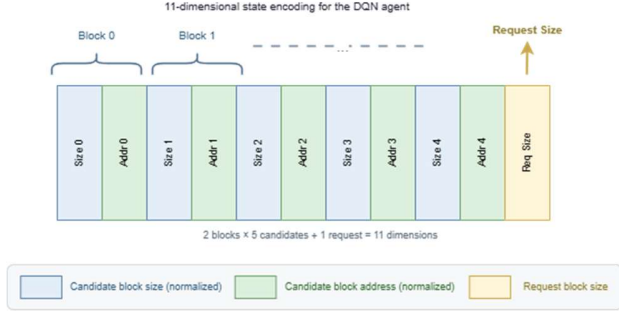


Fig. 2. State vector composition

2) Action Space. The action space is $A = \{0, 1, \dots, k-1\}$, comprising $k = 5$ discrete actions. Action i represents selecting the i -th candidate block for allocation. If the action points to an invalid block (index out of bounds or insufficient block size), the allocation fails, and a fixed negative reward is returned, followed by termination of the current episode.

Compressing the action space to only 5 discrete actions is a key design choice of this paper. Similarly, Mao et al. faced the challenge of action space explosion in DeepRM [5]—if the scheduler could select arbitrary subsets from M pending jobs, the action space would reach the 2^M level. They compressed the action space to $M+1$ by introducing a "void" action and allowing the agent to make multiple decisions within the same time step. A different compression strategy was adopted: utilizing traditional algorithms for candidate block pre-filtering, restricting the scope of RL to the fine-grained decision-making level—this preserves decision flexibility while keeping inference overhead within acceptable bounds.

Additionally, Lim and Maddukuri [6] explored another action space design in related work—having the RL agent directly choose among First-Fit, Best-Fit, or Worst-Fit. While this approach has an extremely small action space (only 3 actions), the decision granularity is too coarse to make fine adjustments at the individual allocation level. The candidate block selection scheme in this paper achieves a better balance between decision granularity and action space size.

3) Reward Function. The reward function is one of the most critical components in RL training; it is essentially the only communication language between the agent and the environment. A good reward function should possess the following properties: (1) Reward continuity—rewards should grow smoothly as the state improves; (2) Fragmentation penalization—rewards should decrease when free space is cut into fragments; (3) Smooth stability—the function should not be overly sensitive to individual allocation/deallocation operations; (4) Normalization—always remaining within a bounded range to avoid gradient explosion.

Inspired by the Herfindahl-Hirschman Index (HHI) from economics, we design the following reward function (1):

$$R = \sum_{f \in F} \left(\frac{I_f}{S_{total}} \right)^2 \quad (1)$$

where F is the free block set, I_f is the size of a single free block, and $S_{total} = \sum I_f$ is the total free space. The desirable properties of

this reward function can be illustrated through several extreme cases:

Ideal case: If only one contiguous free block exists (zero fragmentation), then $R = 1^2 = 1$, achieving maximum reward.

Uniform fragmentation: If space is evenly divided into N equal fragments, then $R = N \cdot (1/N)^2 = 1/N$. The more fragments (larger N), the lower the reward.

Empty memory: If total free space is 0 (memory exhausted), then $R = 0$ by definition.

Value range: $R \in [0, 1]$, providing natural boundedness and smoothness.

Rather than directly optimizing fragmentation rate or occupancy, we optimize a "health indicator" (free space concentration) that is highly correlated with them. Through the recursive property of the Bellman equation, the Agent can learn to maintain a long-term healthy memory layout based solely on this "current health" reward.

4) Invalid Action Handling. When an invalid action is selected, the environment returns a reward of -1.0 and marks the episode as done. The absolute value of this negative reward exceeds the typical range of normal rewards $[0, 1]$, thereby strongly suppressing the tendency to select invalid actions during training.

C. Training Algorithm

1) Network Architecture. A lightweight three-layer fully connected network (MLP) was adopted as the Q function approximator to maintain extremely low inference latency: Input layer: 11 dimensions ($2k+1$) = state vector; Hidden layer 1: 128 dimensions, ReLU activation; Hidden layer 2: 128 dimensions, ReLU activation; Output layer: 5 dimensions (k) = Q-values for each candidate block.

2) DQN Training Algorithm. The training process uses the standard DQN algorithm [2], combining experience replay and target network as two key techniques. The complete procedure is shown in Algorithm 1.

Algorithm 1: DQN Training Algorithm

Hyperparameters: batch size $B=64$, $\gamma=0.99$, replay buffer capacity $C=10000$, target network update interval $T=50$

- 1: Initialize experience replay buffer $D \leftarrow \emptyset$
- 2: Randomly initialize Q network Q_θ , target network $Q_{\theta^-} \leftarrow Q_\theta$
- 3: for episode $e = 1$ to E do
- 4: $s \leftarrow \text{env.reset}()$
- 5: while not done do
- 6: With probability ϵ select random action a , otherwise $a = \arg\max_a Q_\theta(s, a)$
- 7: Execute a , observe r, s', done
- 8: Store $(s, a, r, s', \text{done})$ to D
- 9: $s \leftarrow s'$
- 10: if $|D| \geq B$ then
- 11: Sample batch $\{(s_j, a_j, r_j, s'_j, \text{done}_j)\}$ from D
- 12: $y_j = r_j + \gamma \cdot (1 - \text{done}_j) \cdot \max_{a'} Q_{\theta^-}(s'_j, a')$
- 13: $L = (1/B) \sum \text{SmoothL1}(Q_\theta(s_j, a_j) - y_j)$
- 14: Backpropagate to update $\theta \leftarrow \theta + \alpha \nabla_\theta L$
- 15: end if

```

16: end while
17:  $\epsilon \leftarrow \max(\epsilon \cdot \epsilon\_decay, \epsilon\_min)$ 
18: if  $e \bmod T\_target = 0$  then  $\theta^- \leftarrow \theta$ 
19: end for

```

D. Request Distribution Modeling

The distribution of training data is crucial for the generalization ability of RL models. In actual programs, the size distribution of memory allocation requests typically exhibits significant skewness: small allocations (e.g., 8–64 bytes) are extremely frequent, while large allocations occur occasionally. This distribution is closer to a log-normal distribution than a symmetric normal distribution. Based on this observation, the training environment in this paper uses a truncated log-normal distribution (Equation (2)) to generate allocation requests:

$$\text{size} \sim \text{clip}(\text{longnormal}(\mu = \ln 32, \delta = 0.9), 1, 512) \quad (2)$$

This distribution is centered at 32 bytes (median), with $\sigma=0.9$ controlling the degree of skewness, truncated to the [1, 512] interval to simulate the typical range of malloc requests in practical systems. The overall process is illustrated in Fig. 3.

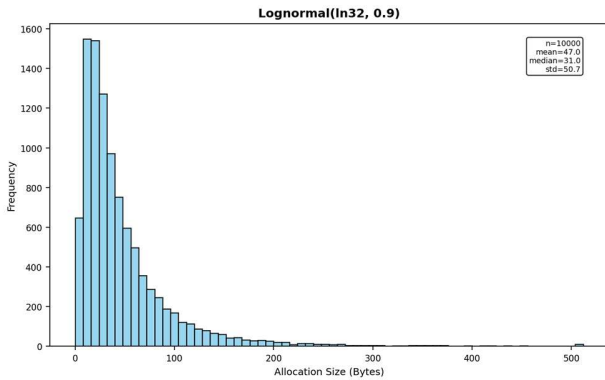


Fig. 3. Training request distribution—Histogram of Lognormal(ln32, 0.9)

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

Environment Parameters. The total memory size is set to $M = 4096$, with the number of candidate blocks $k = 5$. In training mode, the release probability is 0.3 (after each allocation, there is a 30% probability of randomly deallocating an allocated block).

Comparison Baselines. The proposed approach was compared against three classical heuristic algorithms: Best-Fit (traverses all free blocks, selects the one that satisfies the request with the minimum remaining fragment), First-Fit (searches from the lowest address, selects the first free block that satisfies the request), and Worst-Fit (traverses all free blocks, selects the largest one that satisfies the request).

Evaluation Metrics. the four metrics defined in Section II: Occupancy, Duration, Fragmentation, and HHI was used as evaluation metrics.

Evaluation Protocol. To ensure fair comparison, we evaluate all policies using the same request sequence in each test round.

Each round first generates a fixed sequence containing 200 allocation requests and random deallocation events, then runs simulations with each policy on that sequence and records the metrics.

B. Request Distribution Robustness Test

An inherent limitation of traditional heuristic algorithms is the inability to adapt to workload changes. This experiment aims to verify: Can an RL policy, after training on a single distribution, maintain its advantages on unseen distributions? We train the DQN Agent on the truncated log-normal distribution Lognormal(ln32, 0.9), then conduct 1000 rounds of testing on the training distribution itself and three distributions not seen during training, while also testing three traditional heuristic algorithms as controls. The RL policy remains fixed throughout all tests (no further training).

The four test distributions are:

- (1) Lognormal (ln32, 0.9) (training distribution): Centered at 32 bytes, skewed toward small requests, used as baseline reference.
- (2) Lognormal (ln128, 0.7): Request center shifted to larger sizes at 128 bytes, with smaller σ , simulating workloads dominated by large allocations.
- (3) Uniform (1, 512): Unbiased uniform distribution with completely random request sizes.
- (4) Bimodal (70% small + 30% large): Bimodal distribution—70% probability of generating small requests ($\mu=16$), 30% probability of generating large requests ($\mu=256$), simulating the shock scenario of "sudden large requests."

The result show in Fig. 4 and Fig. 5. The experimental results reveal the following key findings:

- (1) DQN approaches Best-Fit on the training distribution and consistently outperforms First-Fit. On the training distribution Lognormal(32), the DQN Agent's occupancy (0.9527) is nearly identical to First-Fit (0.9520), but its fragmentation rate (0.6521) is approximately 4.5% lower than First-Fit (0.6828). In terms of HHI, DQN Agent (0.8165) significantly exceeds both First-Fit (0.7942) and Best-Fit (0.7548), indicating that the learned policy tends to maintain free space concentration while achieving utilization comparable to First-Fit.
- (2) After transferring to new distributions, DQN's advantages over First-Fit are maintained or even amplified. Although DQN was trained only on Lognormal (32), it maintains advantages over First-Fit on all three new distributions. For example, on the Bimodal distribution, DQN occupancy (0.9213) is significantly higher than First-Fit (0.9124), with a gap of approximately 1%. This validates that the RL policy has not merely memorized the statistical properties of the training distribution but has learned a transferable "maintain concentration" behavioral pattern.
- (3) Best-Fit remains the gold standard for occupancy and fragmentation. Across all four distributions, Best-Fit ranks first or near-first in occupancy, duration, and fragmentation. Best-Fit's only weakness is its relatively low HHI, which is a direct manifestation of its greedy nature: to minimize single-step waste, it freely cuts free space into more small blocks.

(4) DQN vs Best-Fit: The trade-off between fragmentation and concentration. Best-Fit trades lower HHI for the lowest fragmentation rate and highest occupancy; DQN trades slightly higher fragmentation for higher HHI. This difference is most extreme on the Bimodal distribution: Best-Fit's HHI is only 0.5076, while DQN achieves 0.5720 (12.7% higher), but Best-Fit's fragmentation rate is 15.8% lower.

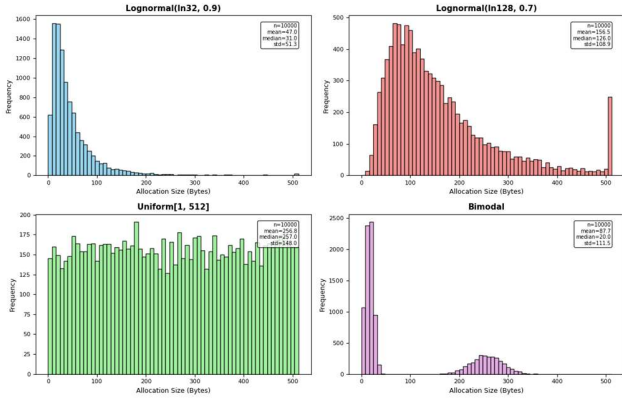


Fig. 4. Request size histogram distribution

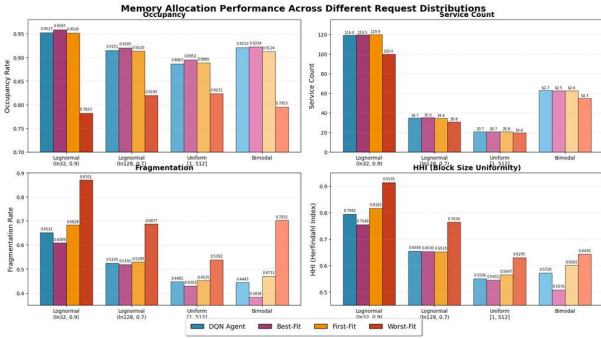


Fig. 5. Performance comparison histograms of four strategies across four distributions

(5) Worst-Fit consistently underperforms across all distributions. Worst-Fit has the worst occupancy, lowest duration, and highest fragmentation rate across all four distributions. Its HHI is consistently the highest—once again demonstrating that pursuing free space concentration alone does not yield good overall performance.

V. DISCUSSION AND CONCLUSION

As Wilson et al. pointed out, real program allocation patterns exhibit significant locality and periodicity, which may impose

different requirements on RL policy learning and generalization [7,8]. This paper focuses on evaluating decision quality rather than execution efficiency. In a practical memory allocator, each allocation operation requires a neural network forward pass, whose overhead is greater in absolute terms than traditional linked-list searches. However, given that the candidate block set has already been pre-processed by the underlying search, the additional inference overhead is highly controllable. Furthermore, the application of techniques such as model pruning and quantization is expected to further reduce inference latency.

This paper explored the feasibility of applying deep reinforcement learning to the dynamic memory allocation problem. We formulated memory allocation as a Markov Decision Process, proposed a DQN framework based on candidate block selection, and designed a reward function based on the Herfindahl-Hirschman Index to guide the Agent in learning to maintain the concentration of free memory space. Distribution robustness tests demonstrate that the RL policy exhibits generalization ability superior to fixed heuristic algorithms on unseen request distributions. The experimental results preliminarily validate the feasibility of RL in the memory allocation domain and reveal the potential of RL in long-term memory health maintenance through implicit lookahead capability.

REFERENCES

- [1] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, "Dynamic storage allocation: A survey and critical review," in Proc. Int'l Workshop Memory Management (IWMM), Kinross, UK, 1995, pp. 1-116.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529-533, 2015.
- [3] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [4] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3-4, pp. 279-292, 1992.
- [5] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in Proc. 15th ACM Workshop on Hot Topics in Networks (HotNets), Atlanta, GA, USA, 2016, pp. 50-56.
- [6] A. Lim and A. Maddukuri, "Reinforcement learning for dynamic memory allocation," arXiv preprint arXiv:2410.15492, 2024.
- [7] Kim T, Kim H. LWMalloc: A lightweight dynamic memory allocator for resource-constrained environments. IEEE Internet of Things Journal, 2025, 12(12): 19532-19548.
- [8] Y. Gu, Z. Liu, S. Dai, C. Liu, Y. Wang, S. Wang, G. Theodoropoulos, and L. Cheng, "Deep reinforcement learning for job scheduling and resource management in cloud computing: An algorithm-level review," arXiv preprint arXiv:2501.01007, 2025.